Shrink, Detect, Fix

Measuring and Reducing Code Hallucinations in Quantized Local LLMs
Research plan · October 2026

1. The research in plain words

Large language models (LLMs) that write code sometimes **hallucinate**: they import libraries that do not exist, call functions that are not real, use variables they never defined, or write code that runs but gives wrong answers. People with small GPUs (like an 8 GB card) run **quantized** models: compressed to 8, 6, 4, 3 or 2 bits so they fit in memory. We know compression barely changes the overall pass rate at 4-bit, but nobody has checked whether it makes models **hallucinate more**, whether the usual hallucination alarms still work after compression, or how to fix it cheaply.

This project extends Saha et al. (2025), who checked LLM hallucination by asking questions with typos and rewording and measuring whether answers stay consistent. Their weakness: they never checked whether consistent answers were actually correct. Code fixes that problem, because unit tests tell us automatically whether the code is right.

The story of the paper: shrinking a model makes it hallucinate more → we **measure** how much → we **detect** when it happens without an answer key → we **fix** it cheaply on the same small GPU.

The four parts

Part	Question	What we do
A · Hallucination Score	How much does a model hallucinate?	A report card per model: % fake libraries, % fake functions, % undefined variables, % code that runs but is wrong, % that breaks under typos. Everything is checked automatically. This is the ruler for all other parts.
B · Shrinking	Does quantization cause more hallucination?	Qwen2.5-Coder 1.5B / 3B / 7B at 8, 6, 4, 3, 2 bits, measured with the Part A score. Also count “flips”: problems the bigger-bit model solved but the smaller one fails.
C · Detecting	Do hallucination alarms still work after shrinking?	Compare detectors that need no answer key: model confidence (token entropy), agreement across repeated answers, agreement across typo/reworded prompts, static check for fake imports. Measure AUROC at each bit-width.
D · Fixing	Can we reduce hallucination cheaply?	The best detector from Part C becomes a gatekeeper that decides when to spend extra effort (see below).

The fix (Part D)

Step	What happens	Cost
1. Check imports and functions	Verify every library and function exists; if not, tell the model what is wrong and retry.	Very cheap
2. Gatekeeper	If the detector says the model is confident, accept the answer.	Free
3. Several versions	Only for unsure cases: generate 5 versions, run them on the same inputs, keep the one most agree on.	Medium
4. Say “not sure”	If the versions still disagree, refuse instead of returning bad code.	Free

Headline question: “With only 8 GB, is it better to run a bigger model shrunk hard, or a smaller model with our fix?” For example, Qwen-7B at 3-bit without the fix vs Qwen-3B at 4-bit with the fix, at the same memory and time.

Rules that keep it publishable

- **Fair comparison.** If the fix uses 5 generations, the baseline also gets 5 (plain resampling + majority vote).
- **Test the perturbation idea honestly.** Compare 5 answers to the same prompt vs 5 answers to altered prompts. A 2026 study found typo prompts can be worse than plain resampling; report whatever you find.
- **Don't claim the pieces as new.** Import checking, execution voting and abstention already exist. What is new: the combination, the gatekeeper, and the quantized local-model setting.
- **Biggest novelty:** whether detectors that work at 16-bit still work at 4-bit (no paper has tested this).

Size, timeline and where to submit

Phase	Work	Time
1	Setup: llama.cpp, sandbox, HumanEval+ / MBPP+, Part A score	2–3 weeks
2	Quantization sweep: Parts B and C	3–4 weeks (GPU runs in background)
3	Build the fix (Part D) + fair baselines	3–4 weeks
4	Statistics and writing	2–3 weeks

Total ≈ 3 months. Full paper (~8 pages ACL, 10+ pages SE venues). Targets: **ACL via ARR, 4 Jan 2027** (tight); a software-engineering journal such as **EMSE, IST or JSS** (rolling); safety option: Parts A + B + C to an ACL 2027 workshop (~April 2027), then extend. Check each venue's call for papers; deadlines here were found by web search in Oct 2026.

2. Step-by-step plan

Each step is short on purpose. Do them in order; steps 8–11 can overlap while the GPU runs in the background.

Step 1 · Read the key papers (week 1)

Start with the ★ papers in Section 3. For each, write 3 lines: what they did, their numbers, how your work differs. Pay most attention to Wang & Zhu (prompt paraphrasing for code) and the quantization papers, because reviewers will compare you to them.

Step 2 · Set up your machine

Install llama.cpp and run llama-server with parallel slots. Download Qwen2.5-Coder-Instruct 1.5B, 3B and 7B as GGUF files at Q8_0, Q6_K, Q4_K_M, Q3_K_M and Q2_K (7B: Q6_K and below, since Q8 does not fit in 8 GB). Confirm you can get token probabilities (n_probs) back from the server.

Step 3 · Build a safe sandbox

Generated code can loop forever or delete files. Run it in Docker with no network (--network none), a time limit and a memory limit. Never pip install a package name the model suggested: attackers register fake names (“slopsquatting”).

Step 4 · Load the benchmarks

Use HumanEval+ and MBPP+ from EvalPlus (stronger tests). Add a recent LiveCodeBench slice (problems newer than the model's training data) and BigCodeBench-Hard (148 library-heavy tasks where fake APIs actually show up). Save every problem with an ID so all runs are comparable.

Step 5 · Build the hallucination checkers (Part A)

Write one checker per type: does each import exist (try importing in a clean environment), does each called function/attribute exist (introspection), undefined names (pyflakes), runtime error type, and pass/fail on tests. Output one row per answer with all flags.

Step 6 · Build the perturbations

Create 9 altered versions of each problem: 3 character-level (typos), 3 word-level (synonyms, drop/add filler words), 3 sentence-level (paraphrase, add irrelevant handle, restructure). Change only the description, never the function name. Hand-check ~50 to make sure the meaning is unchanged.

Step 7 · Run a small pilot

One model (Qwen-3B) at two bit-widths (Q8 and Q4) on HumanEval+ only. Check the whole pipeline end to end, measure time per answer, and fix bugs now. This is also enough for an early result to show a supervisor or the original authors.

Step 8 · Part B: the quantization sweep

Run every model × bit-width on all benchmarks with greedy decoding. Compute the Hallucination Score for each and the “flips” (problems solved at higher precision but failed at lower). Plot score vs bits per weight for each model size.

Step 9 · Part C: test the detectors

For every answer compute: token entropy (code tokens only), agreement across 5 samples of the same prompt, agreement across 5 altered prompts (run the programs on the same inputs and compare outputs), and the static import check. Measure AUROC against the real test result, per bit-width.

Step 10 · Part D: build the fix

Implement the 4 steps: import/function check with retry, gatekeeper (best detector from step 9), 5-version execution vote for unsure cases, and “not sure” when they disagree. Choose the gatekeeper threshold on a held-out half of the problems, then test on the other half.

Step 11 · Fair baselines and the headline question

Compare against greedy, plain resampling + majority vote, and self-repair, all at the same number of generations. Draw risk–coverage curves. Answer the headline question: bigger model at low bits vs smaller model at higher bits + fix, at equal memory and time.

Step 12 · Statistics

Use 3 random seeds, bootstrap confidence intervals for every number, McNemar's test for pass/fail differences and a paired bootstrap for AUROC. Define every average exactly (Saha et al.'s unclear Figure 3 is the mistake to avoid).

Step 13 · Write, check and submit

Check every number you cite against the actual PDF (many came from search snippets). Post a preprint on arXiv, release code and all generations, and submit to ARR (4 Jan 2027) or a journal. Re-search arXiv the week before submitting, since this area moves fast.

3. Related papers you must read

★ = read first. Links are to arXiv or the official page. Titles for some 2025–2026 papers are described rather than quoted; open the link to get the exact title before citing.

The starting point and your closest competitors

	Paper	Why read it	Link
★	Saha, Gupta, Chatterjee, Banerjee (2025). You believe your LLM is not delusional? Think again! Discover Data 3:20	The paper you are extending.	doi.org/10.1007/s44248-025-00041-7
★	Wang & Zhu (2024). Validating LLM-Generated Programs with Metamorphic Prompt Testing; ICSE 2026 NIER version: Automatic Validation of LLM-Generated Code with Prompt Paraphrasing	Closest prior work: paraphrase prompts, run the code, cross-check. You must compare against it.	arxiv.org/abs/2406.06864
★	Semantic Triangulation (OOPSLA 2026)	Reports beating paraphrase-based testing; reviewers will ask about it.	arxiv.org/abs/2511.12288

Measuring code hallucination (Part A)

	Paper	Why read it	Link
★	Tian et al. (AAAI 2025). CodeHalu: Investigating Code Hallucinations in LLMs via Execution-based Verification	Automatic per-model hallucination types by running code. Nearest to your score.	arxiv.org/abs/2405.00253
	Liu et al. (2024). Exploring and Evaluating Hallucinations in LLM-Powered Code Generation (HalluCode)	Hand-built taxonomy of code hallucinations.	arxiv.org/abs/2404.00971
	Zhang et al. (ISSTA 2025). LLM Hallucinations in Practical Code Generation	Real-world hallucination categories on CoderEval.	arxiv.org/abs/2409.20550
★	Spracklen et al. (USENIX Security 2025). We Have a Package for You! (package hallucination)	19.7% of suggested packages don't exist; open models much worse.	github.com/Spracks/PackageHallucination
	AST + introspection API-hallucination detector (2026)	How to check fake functions automatically (100% precision, 87.6% recall).	arxiv.org/abs/2601.19106
★	Liu et al. (NeurIPS 2023). Is Your Code Generated by ChatGPT Really Correct? (EvalPlus)	HumanEval+ / MBPP+: the benchmarks you will use.	github.com/evalplus/evalplus
	Wang et al. (ACL 2023). ReCode: Robustness Evaluation of Code Generation Models	Prompt perturbations for code and the Robust Drop metric.	aclanthology.org/2023.acl-long.773
	NLPerturbator (TOSEM): natural-language perturbations for code generation	18 perturbation categories; useful for building yours.	arxiv.org/abs/2406.19783
	Riddell et al. (ACL 2024). Quantifying Contamination in Evaluating Code Generation Capabilities of Language Models	Why HumanEval/MBPP alone is not enough; use LiveCodeBench too.	aclanthology.org/2024.acl-long.761

Detecting hallucination without tests (Part C)

	Paper	Why read it	Link
★	Farquhar et al. (Nature 2024). Detecting hallucinations in large language models using semantic entropy	The standard agreement-based detector.	nature.com/articles/s41586-024-07421-0
	Manakul et al. (EMNLP 2023). SelfCheckGPT	Same-prompt resampling consistency, the baseline Saha et al. compared to.	arxiv.org/abs/2303.08896
★	Spiess et al. (ICSE 2025). Calibration and Correctness of Language Models for Code	How well token confidence predicts code correctness.	software-lab.org/publications/icse2025_calibration.pdf
	Sharma & David (2025): symbolic-execution semantic entropy for code	Execution-based clustering for code uncertainty.	arxiv.org/abs/2502.11620
	Code-token entropy + execution ensemble (2026)	AUROC 0.696 → 0.776; score only code tokens, not comments.	arxiv.org/abs/2606.09577
	Functional entropy for code (2026)	Shows text-similarity clustering fails for code; justifies execution agreement.	arxiv.org/abs/2605.28500

Fixing hallucination (Part D)

	Paper	Why read it	Link
★	Chen et al. (ICLR 2023). CodeT: Code Generation with Generated Tests	Execution-agreement selection baseline (HumanEval 47.0% → 65.8%).	arxiv.org/abs/2207.10397
	Shi et al. (EMNLP 2022). Natural Language to Code Translation with Execution (MBR-exec)	“Keep the answer most versions agree on”: your voting step.	arxiv.org/abs/2204.11454
★	Olausson et al. (ICLR 2024). Is Self-Repair a Silver Bullet for Code Generation?	How to compare methods fairly at equal budget.	arxiv.org/abs/2306.09896
	Amazon (2024). On Mitigating Code LLM Hallucinations with API Documentation (CloudAPIBench)	Fixing fake APIs with documentation; only retrieve when needed.	arxiv.org/abs/2407.09726
★	Execution clustering with abstention (2025)	Error of returned answers drops ~65% → 2% by saying “not sure”.	arxiv.org/abs/2506.11021
	CodeRefuser (2026): calibrated abstention for code	Abstention with a statistical guarantee; strong baseline for step 4.	arxiv.org/abs/2605.17029
★	Typo sampling vs temperature sampling at matched budget (2026)	Typo prompts did worse than plain resampling; the risk your paper must face.	arxiv.org/abs/2606.15521
	Self-repair vs resampling on small 4-bit models (2026)	On Qwen2.5-Coder 1.5–7B Q4_K_M, blind resampling beats self-repair.	arxiv.org/abs/2607.26117

Quantization (Part B)

	Paper	Why read it	Link
★	Secure or Suspect? (2025): quantization and package hallucination	The only code-hallucination × quantization study; 4-bit made it worse.	arxiv.org/abs/2512.08213
★	Kurtic et al. (2024). “Give Me BF16 or Give Me Death”? Accuracy-Performance Trade-Offs in LLM Quantization	8-bit ≈ 99.9%, 4-bit ≈ 98.9% of HumanEval accuracy.	arxiv.org/abs/2411.02355
	Quantizing LLMs for code generation: a differentiated replication (2025)	4-bit is the practical limit for code; 2-bit collapses.	arxiv.org/abs/2503.07103
	Hong et al. (ICML 2024). Decoding Compressed Trust	4-bit keeps trustworthiness, 3-bit loses it.	arxiv.org/abs/2403.15447
★	The “illusion of equivalency” in quantized models (2026)	Same accuracy, but many individual answers flip; supports your “flips” metric.	arxiv.org/abs/2607.08734
	Qwen Team (2024). Qwen2.5-Coder Technical Report	The models you will run.	arxiv.org/abs/2409.12186

Tools

	Paper	Why read it	Link
	llama.cpp server documentation	Running GGUF models, parallel slots, token probabilities.	github.com/ggml-org/llama.cpp
	LiveCodeBench and BigCodeBench	Newer and library-heavy benchmarks.	github.com/LiveCodeBench/LiveCodeBench · github.com/bigcode-project/bigcodebench

Before citing: many numbers in this plan came from search-result summaries, not the full papers. Open each PDF and check the exact figures and titles before they go into your paper.